МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Языки программирования

Классы в Python

Руководитель _____ А.В. Проскурин
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151430 _____ В.Т. Гордов
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

ЦЕЛЬ РАБОТЫ

Получение практических навыков разработки и отладки программ, использующих экземпляры собственного класса.

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться с общей постановкой задачи.
2. Ознакомится с вариантом задания – соответствует вашему номеру в списке группы (при нехватке заданий вариант задания вычисляется как номер_в_списке_группы - количество_заданий).
3. Разработать класс согласно варианту задания.
4. Написать и отладить программу на подготовленных наборах тестовых данных.
5. Подготовить отчет по лабораторной работе. Отчет должен включать в себя:
   - титульный лист;
   - цель лабораторной работы;
   - постановку задачи;
   - текст программы с комментариями;
   - демонстрацию работы программы (Снимки экрана при выполнении действий программы с описанием).
   - краткие ответы на контрольные вопросы;
   - выводы по лабораторной работе.
6. Защитить лабораторную работу перед преподавателем.

ПОСТАНОВКА ЗАДАЧИ

Необходимо разработать программу на языке Python, реализующую задачу ранее решенную на языке C++ в лабораторной работе №1.

Программ должна формировать и обрабатывать **список экземпляров класса** (согласно варианту задания).

В реализуемой программе необходимо предусмотреть возможность **демонстрации методов** созданного класса.

В зависимости от оценки, на которую вы претендуете, необходимо выполнить следующие задания (Для каждой следующей оценки нужно выполнить **ВСЕ** предыдущие задания, если обратное не указано явно):

Оценка	Общее задание
Удовлетворительно	1. Определить класс (согласно варианту задания) (обозначим его Sl). Все поля должны быть закрытыми (начинаться с двух символов подчёркивания).
	2. Написать метод заполнения полей экземпляров класса данными с клавиатуры.
	3. Написать метод вывода полей экземпляров класса на экран.
	4. Написать методы по проверке значений двух полей (согласно варианту задания – два любых на ваш выбор). **По одному методу** на проверку **каждого поля**. Каждый метод принимает значение для сравнения (или два, если проверяется вхождение в диапазон) и возвращает булево значение.
	5. В main продемонстрировать работу класса. Нужно создать экземпляр или список экземпляров класса. Заполнить его с клавиатуры и вывести на экран. Также необходимо реализовать вывод на экран экземпляров, удовлетворяющих заданным условиям (использовать методы из задания 4).
Хорошо	6. Реализовать оставшийся метод проверки значения поля (согласно варианту задания). В main необходимо вывести на экран экземпляры, удовлетворяющие заданному условию проверки.
	7. Написать метод заполнения полей экземпляров класса данными из файла. Использовать его в main.
	8. Написать метод вывода в файл полей экземпляров класса. В main записать в файл экземпляры, полученные после выборки (от реализации задач 5, 6).
Отлично	9. Реализовать класс (обозначим его Ms), полем которого будет список экземпляров класса Sl (задание 1). Конструкции из main для заполнения списка, его вывода и поиска нужных экземпляров (заданий 5–8) оформить как методы класса Ms (в этом случае можно писать их сразу в классе Ms).

Вариант №5.
Класс House (Квартира):
Поля: Адрес, Тип здания, Этаж, Количество комнат, Площадь.
Реализовать выборки:
а) списка квартир, имеющих заданное число комнат;
б) списка квартир, имеющих заданное число комнат и расположенных в указанном диапазоне этажей;
в) списка квартир, имеющих площадь больше заданной.
Реализовать сортировку массива объектов класса на основе:
- площади.

ХОД РАБОТЫ

Файл **header.py**:

```python
class House:
    """
    Класс, представляющий квартиру.
    Поля:
    - `address (str)`: Адрес квартиры.
    - `building_type (str)`: Тип здания.
    - `floor (int)`: Этаж.
    - `rooms (int)`: Количество комнат.
    - `area (float)`: Площадь в кв. м.
    """

    def __init__(self, address="", building_type="", floor=1, rooms=1, area=0.0):
        self.__address = address.strip()
        self.__building_type = building_type.strip()
        self.__floor = floor
        self.__rooms = rooms
        self.__area = float(area)

    # Геттеры для полей
    def get_address(self):
        return self.__address

    def get_building_type(self):
        return self.__building_type

    def get_floor(self):
        return self.__floor

    def get_rooms(self):
        return self.__rooms

    def get_area(self):
        return self.__area

    # Методы ввода и вывода
    def input_data(self, prompt="Введите данные о квартире:"):
        """
        Метод ввода данных с клавиатуры.
        """
        print(prompt)
        address = input("Адрес: ")
        building_type = input("Тип здания: ")
        floor = int(input("Этаж: "))
        rooms = int(input("Количество комнат: "))
        area = float(input("Площадь (кв. м.): "))

        self.__address = address.strip()
        self.__building_type = building_type.strip()
        self.__floor = floor
        self.__rooms = rooms
        self.__area = float(area)

    def input_from_file(self, file_data):
        """
        Метод ввода данных из файла.
        """
        # Читаем строку и делим на 5 подстрок
        parts = file_data.strip().split(',')
        if len(parts) != 5:
            raise ValueError("Строка не содержит 5 значений, разделённых запятыми.")
```

```python
            raise ValueError("Строка не содержит 5 значений, разделённых запятыми.")

        address = parts[0]
        building_type = parts[1]
        floor = int(parts[2])
        rooms = int(parts[3])
        area = float(parts[4])

        self.__address = address.strip()
        self.__building_type = building_type.strip()
        self.__floor = floor
        self.__rooms = rooms
        self.__area = float(area)

    def output(self):
        """
        Метод вывода данных на экран.
        """
        print(f"Адрес: {self.__address}\n Тип: {self.__building_type}\n Этаж: {self.__floor}\n Комнат: {self.__rooms}\n Площадь: {self.__area:.2f} кв. м.")

    def to_line(self):
        """
        Вспомогательный метод для сериализации в строку для .txt.
        Используется для записи в файл.
        """
        return f"{self.__address},{self.__building_type},{self.__floor},{self.__rooms},{self.__area:.2f}"

    # Методы проверки
    def has_n_rooms(self, n):
        """Проверяет, равно ли количество комнат заданному числу."""
        return self.__rooms == n

    def is_on_floor_range(self, min_floor, max_floor):
        """Проверяет, находится ли этаж в заданном диапазоне."""
        return min_floor <= self.__floor <= max_floor

    def area_greater_than(self, min_area):
        """Проверяет, больше ли площадь заданной."""
        return self.__area > min_area

class Ms:
    """
    Класс-контейнер для списка квартир (экземпляров класса House).
    """
    def __init__(self):
        self.__houses = []

    def add_house(self, house):
        """Добавляет квартиру в список."""
        self.__houses.append(house)

    def input_houses_from_console(self):
        """Ввод квартир с консоли."""
        n = int(input("Сколько квартир добавить? "))
        for _ in range(n):
            h = House() # Создаём с пустыми значениями по умолчанию
            h.input_data()
            self.add_house(h)
```

```python
            self.add_house(h)
            print("Квартира добавлена.")

    def load_from_file(self, filename):
        """
        Загружает список квартир из текстового файла.
        Формат: Address,Type,Floor,Rooms,Area
        """
        with open(filename, 'r', encoding='utf-8') as f:
            lines = f.readlines()
        self.__houses = [] # Очищаем список квартир перед чтением
        for line in lines:
            if line.strip(): # Не будет считывать пустые строки
                h = House()
                h.input_from_file(line)
                self.add_house(h)
        print(f"Данные загружены из {filename}")

    def save_to_file(self, filename):
        """
        Сохраняет список квартир в текстовый файл.
        Формат: Address,Type,Floor,Rooms,Area
        """
        with open(filename, 'w', encoding='utf-8') as f:
            for house in self.__houses:
                f.write(house.to_line() + "\n")
        print(f"Данные сохранены в {filename}")

    def display_houses(self):
        """Выводит все квартиры на экран."""
        if not self.__houses:
            print("Список квартир пуст.")
            return
        for i, house in enumerate(self.__houses):
            print(f"--- Квартира №{i+1} ---")
            house.output()

    # Выборки данных
    def select_by_rooms(self, n):
        """Список квартир с заданным числом комнат."""
        return [h for h in self.__houses if h.has_n_rooms(n)]

    def select_by_rooms_and_floor_range(self, n, min_floor, max_floor):
        """Список квартир с заданным числом комнат и в диапазоне этажей."""
        return [h for h in self.__houses if h.has_n_rooms(n) and h.is_on_floor_range(min_floor, max_floor)]

    def select_by_area_greater_than(self, min_area):
        """Список квартир с площадью больше заданной."""
        return [h for h in self.__houses if h.area_greater_than(min_area)]

    # Сортировка данных
    def sort_by_area(self):
        """Сортирует квартиры по площади."""
        self.__houses.sort(key=lambda h: h.get_area())
```

Файл **main.py**:

```python
from header import Ms

def main():
    container = Ms()
    filename = "apartments.txt"

    while True:
        print("\n--- Система управления квартирами ---")
        print("1. Добавить квартиры с консоли")
        print("2. Загрузить квартиры из файла")
        print("3. Сохранить квартиры в файл")
        print("4. Показать все квартиры")
        print("5. Выбрать квартиры по количеству комнат")
        print("6. Выбрать квартиры по комнатам и диапазону этажей")
        print("7. Выбрать квартиры по площади больше заданной")
        print("8. Отсортировать квартиры по площади")
        print("0. Выйти")
        choice = input("Выберите действие: ")

        if choice == '1':
            container.input_houses_from_console()
        elif choice == '2':
            container.load_from_file(filename)
        elif choice == '3':
            container.save_to_file(filename)
        elif choice == '4':
            container.display_houses()
        elif choice == '5':
            try:
                n = int(input("Введите количество комнат: "))
                selected = container.select_by_rooms(n)
                print(f"\n--- Квартиры с {n} комнатами ---")
                for h in selected:
                    h.output()
            except ValueError:
                print("Неверный ввод для количества комнат.")
        elif choice == '6':
            try:
                n = int(input("Введите количество комнат: "))
                min_f = int(input("Введите минимальный этаж: "))
                max_f = int(input("Введите максимальный этаж: "))
                selected = container.select_by_rooms_and_floor_range(n, min_f, max_f)
                print(f"\n--- Квартиры с {n} комнатами на этажах {min_f}-{max_f} ---")
                for h in selected:
                    h.output()
            except ValueError:
                print("Неверный ввод для этажей или количества комнат.")
        elif choice == '7':
            try:
                min_a = float(input("Введите минимальную площадь: "))
                selected = container.select_by_area_greater_than(min_a)
                print(f"\n--- Квартиры с площадью > {min_a} кв. м. ---")
                for h in selected:
                    h.output()
            except ValueError:
                print("Неверный ввод для площади.")
        elif choice == '8':
            container.sort_by_area()
            print("Квартиры отсортированы по площади.")
        elif choice == '0':
```

```
60            elif choice == '0':
61                print("Выход...")
62                break
63            else:
64                print("Неверный выбор. Пожалуйста, попробуйте снова.")
65
66    if __name__ == "__main__":
67        main()
```

Добавление квартиры в список:

```
--- Система управления квартирами ---
1. Добавить квартиры с консоли
2. Загрузить квартиры из файла
3. Сохранить квартиры в файл
4. Показать все квартиры
5. Выбрать квартиры по количеству комнат
6. Выбрать квартиры по комнатам и диапазону этажей
7. Выбрать квартиры по площади больше заданной
8. Отсортировать квартиры по площади
0. Выйти
Выберите действие: 1
Сколько квартир добавить? 1
Введите данные о квартире:
Адрес: _адрес_
Тип здания: _тип_
Этаж: 1
Количество комнат: 1
Площадь (кв. м.): 25
Квартира добавлена.
```

```
Выберите действие: 4
--- Квартира №1 ---
Адрес: _адрес_
 Тип: _тип_
 Этаж: 1
 Комнат: 1
 Площадь: 25.00 кв. м.
```

Загрузка квартир из файла:

```
2. Загрузить квартиры из файла
3. Сохранить квартиры в файл
4. Показать все квартиры
5. Выбрать квартиры по количеству комнат
6. Выбрать квартиры по комнатам и диапазону этажей
7. Выбрать квартиры по площади больше заданной
8. Отсортировать квартиры по площади
0. Выйти
Выберите действие: 2
Данные загружены из E:/Codespace/C_codespace/2 kurs/3-rd senester/Programming languages/laboratory work/8-th lab/Vlad/apartments.txt
```

```
4. Показать все квартиры
5. Выбрать квартиры по количеству комнат
6. Выбрать квартиры по комнатам и диапазону этажей
7. Выбрать квартиры по площади больше заданной
8. Отсортировать квартиры по площади
0. Выйти
Выберите действие: 4
--- Квартира №1 ---
Адрес: АДРЕС1
 Тип: ТИП1
 Этаж: 2
 Комнат: 3
 Площадь: 30.00 кв. м.
--- Квартира №2 ---
Адрес: АДРЕС2
 Тип: ТИП1
 Этаж: 3
 Комнат: 2
 Площадь: 40.00 кв. м.
--- Квартира №3 ---
Адрес: АДРЕС3
 Тип: ТИП2
 Этаж: 4
 Комнат: 3
 Площадь: 50.00 кв. м.
```

Осуществление первой, второй и третьей выборок:

```
5. Выбрать квартиры по количеству комнат
6. Выбрать квартиры по комнатам и диапазону этажей
7. Выбрать квартиры по площади больше заданной
8. Отсортировать квартиры по площади
0. Выйти
Выберите действие: 5
Введите количество комнат: 3

--- Квартиры с 3 комнатами ---
Адрес: АДРЕС1
 Тип: ТИП1
 Этаж: 2
 Комнат: 3
 Площадь: 30.00 кв. м.
Адрес: АДРЕС3
 Тип: ТИП2
 Этаж: 4
 Комнат: 3
 Площадь: 50.00 кв. м.
```

```
6. Выбрать квартиры по комнатам и диапазону этажей
7. Выбрать квартиры по площади больше заданной
8. Отсортировать квартиры по площади
0. Выйти
Выберите действие: 6
Введите количество комнат: 3
Введите минимальный этаж: 1
Введите максимальный этаж: 3

--- Квартиры с 3 комнатами на этажах 1-3 ---
Адрес: АДРЕС1
 Тип: ТИП1
 Этаж: 2
 Комнат: 3
 Площадь: 30.00 кв. м.
```

```
7. Выбрать квартиры по площади больше заданной
8. Отсортировать квартиры по площади
0. Выйти
Выберите действие: 7
Введите минимальную площадь: 35

--- Квартиры с площадью > 35.0 кв. м. ---
Адрес: АДРЕС2
 Тип: ТИП1
 Этаж: 3
 Комнат: 2
 Площадь: 40.00 кв. м.
Адрес: АДРЕС3
 Тип: ТИП2
 Этаж: 4
 Комнат: 3
 Площадь: 50.00 кв. м.
```

Сортировка квартир по площади:

```
Выберите действие: 4
--- Квартира №1 ---
Адрес: АДРЕС1
 Тип: ТИП1
 Этаж: 2
 Комнат: 3
 Площадь: 50.00 кв. м.
--- Квартира №2 ---
Адрес: АДРЕС2
 Тип: ТИП1
 Этаж: 3
 Комнат: 2
 Площадь: 30.00 кв. м.
--- Квартира №3 ---
Адрес: АДРЕС3
 Тип: ТИП2
 Этаж: 4
 Комнат: 3
 Площадь: 40.00 кв. м.
```

```
8. Отсортировать квартиры по площади
0. Выйти
Выберите действие: 8
Квартиры отсортированы по площади.
```

```
4. Показать все квартиры
5. Выбрать квартиры по количеству комнат
6. Выбрать квартиры по комнатам и диапазону этажей
7. Выбрать квартиры по площади больше заданной
8. Отсортировать квартиры по площади
0. Выйти
Выберите действие: 4
--- Квартира №1 ---
Адрес: АДРЕС2
 Тип: ТИП1
 Этаж: 3
 Комнат: 2
 Площадь: 30.00 кв. м.
--- Квартира №2 ---
Адрес: АДРЕС3
 Тип: ТИП2
 Этаж: 4
 Комнат: 3
 Площадь: 40.00 кв. м.
--- Квартира №3 ---
Адрес: АДРЕС1
 Тип: ТИП1
 Этаж: 2
 Комнат: 3
 Площадь: 50.00 кв. м.
```

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. **В чем состоит сущность методологии объектно-ориентированного программирования?**
   В представлении программы как совокупности объектов, каждый из которых является экземпляром определённого класса. Основные принципы: инкапсуляция, наследование, абстракция и полиморфизм. Это позволяет организовать код в виде взаимодействующих объектов, упрощая разработку, тестирование и поддержку.
2. **Что такое класс? Как выполнить описание класса?**
   Это шаблон для создания объектов, описывающий их свойства (поля) и поведение (методы). Описание производится при определении метода **__init__**.
3. **Что такое экземпляр класса и как его создать в программе?**
   Это конкретный объект, созданный на основе класса. Синтаксис: **obj = class_name (args)**
4. **Что такое поле класса?**
   Переменная, хранящая состояние объекта. Привязывается к экземпляру класса, задаётся при его описании (**__init__**).
5. **Что такое методы класса?**
   Функции, определённые внутри класса и привязанные к его объектам (т.е. описывают их поведение). Первый аргумент всегда **self**, являющийся ссылкой на данный объект.
6. **Как выполняется вызов метода класса?**
   obj.method(self, args)
7. **Какие методы называются конструкторами?**
   Конструктор — это специальный метод, который автоматически вызывается при создании объекта. Определяется как метод **__init__**.
8. **Как выполняется инициализация экземпляров класса?**
   При вызове метода **__init__**. В нём присваиваются начальные значения полей объекта.
9. **Как в классе реализуются инкапсуляция?**
   Реализуется условно (т.е. не регулируется языком программирования функционально) с помощью уровней доступа. **_name** для защищённых полей (не используются напрямую) и **__name** для приватный полей (доступны только внутри класса). Остальные поля — публичные.

ВЫВОДЫ

Я получил практические навыки разработки и отладки программ на языке Python, использующих экземпляры собственного класса.